

UNIVERSIDAD DEL VALLE DE GUATEMALA
FACULTAD DE INGENIERÍA



Nix para todos: Impacto del uso de un lenguaje de propósito general en la usabilidad de Nix

Trabajo de graduación presentado por Daniel Alfredo Rayo Roldán para optar al grado académico en
Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Índice

1. Resumen	1
2. Introducción	2
3. Objetivos	4
3.1. General	4
3.2. Específicos	4
4. Justificación	5
5. Metodología	7
5.1. Confidencialidad y Seguridad	7
5.2. Fase 1: Investigación preliminar	7
5.2.1. Perfil de los participantes	7
5.2.2. Instrumentos	7
5.2.3. Procedimiento	8
5.2.4. Análisis de datos	8
5.3. Fase 2: Desarrollo del eDSL	8
5.4. Fase 3: Evaluación comparativa	8
5.4.1. Perfil de los participantes	8
5.4.2. Instrumentos	8
5.4.3. Diseño experimental	9
5.4.4. Procedimiento	9
5.4.5. Métricas y análisis de datos	9
6. Plan de Trabajo	10
7. Índice preliminar	11
Referencias	12
8. Anexos	15
8.1. Soluciones existentes a NixLang	15
8.2. Interés en Neovim	15

1. Resumen

El problema de llevar *software* de una computadora a otra y que siga funcionando (proceso conocido como Despliegue de *Software*) es un yugo con el que las Ciencias de Computación no han dado una solución definitiva, esencialmente porque para que una pieza de *software* funcione correctamente no solamente depende del código fuente en el que esta escrita, sino del contexto que le rodea (*hardware*, sistema operativo, dependencias, etc.), bajo ese contexto surgen Nix como un manejador de paquetes y sistema de construcción¹ que utiliza un enfoque inspirado en la pureza funcional, donde cada paquete define explícitamente el contexto en el que espera ser construido y ejecutado. Este enfoque ha demostrado poder crear más de 700 mil artefactos binariamente idénticos en diferentes computadoras y contar con el repositorio de paquetes más grande a fecha de este documento. La idea evolucionó al punto de definir el estado casi completo de un sistema operativo mediante un solo archivo de configuración.

Sin embargo, a pesar de sus capacidades prometedoras, Nix no ha gozado de la misma adopción que otras herramientas que abordan los mismos problemas de reproducibilidad como Docker, Conda o VirtualEnv u otros manejadores de paquetes. Las causas de acuerdo a la comunidad son varias: Documentación compleja, errores crípticos, o un lenguaje de programación difícil de dominar; muchos de ellos no siendo problemas técnicos sino de experiencia de uso, y que cae en el rango de estudio del «DX» (Experiencia de Desarrollo por sus siglas en inglés). Este trabajo se enfoca en verificar si proveer una nueva forma de interactuar con Nix a través de un lenguaje de propósito general como Typescript, puede reducir la barrera de entrada para nuevos usuarios y por ende mejorar su DX.

¹Sistemas que automatizan la ejecución de tareas repetitivas, usualmente para crear artefactos de *software* que pueden ser desplegados

2. Introducción

El distribuir *software* de las cocinas de los ingenieros a las mesas de los usuarios finales no ha sido una tarea sencilla [1], los artefactos de *software* se comportan igual a las plantas exóticas cuando son transplantadas a un hábitat diferente al que están acostumbradas: se marchitan [2]. Como las plantas, el *software* «crece y evoluciona» en el *hardware*, sistema operativo y librerías específicas, de la computadora del ingeniero, pero en el momento que esos artefactos son llevados a los ecosistemas extraños, que son los dispositivos de los usuarios finales, el que funcione o no se vuelve una apuesta ante la que no se tiene control... o sí? [2].

El problema anteriormente descrito, es el sujeto de estudio del campo de «Manejo de Configuración de *Software*» (o CSM por sus siglas en inglés), donde se reconoce que la ejecución correcta de *software* no solamente depende de su código fuente, sino del contexto que le rodea [2]. Con los años se ha desarrollado una familia de *software*, llamada **manejadores de paquetes** responsable de modificar el entorno global de las computadoras objetivo para conseguir las condiciones ideales para cada aplicación. Al día de hoy se han convertido en una familia tan variada que se han vuelto una característica diferenciadora en las diferentes distribuciones de Linux o lenguajes de programación [3]. Una corriente opuesta es la **virtualización**, que consiste en empaquetar las aplicaciones junto a los entornos completos que necesitan y ejecutarlas de forma aislada, soluciones de este tipo son muy usadas en servicios de la nube [4].

Sin embargo, como se ilustra en la Figura 1, los manejadores de paquetes lidian con el problema que intentar satisfacer a varias aplicaciones en un entorno global puede llevar a conflictos irresolubles: cuando F00 y BAR dependen de versiones distintas de Node, el entorno global obliga a elegir una sola versión compatible (Node v23.8). La virtualización elimina ese conflicto permitiendo que cada aplicación lleve su propia versión en entornos separados, pero a cambio puede duplicar dependencias compartidas como Clang 19.2 [5], incrementando el consumo de almacenamiento [6], [7].

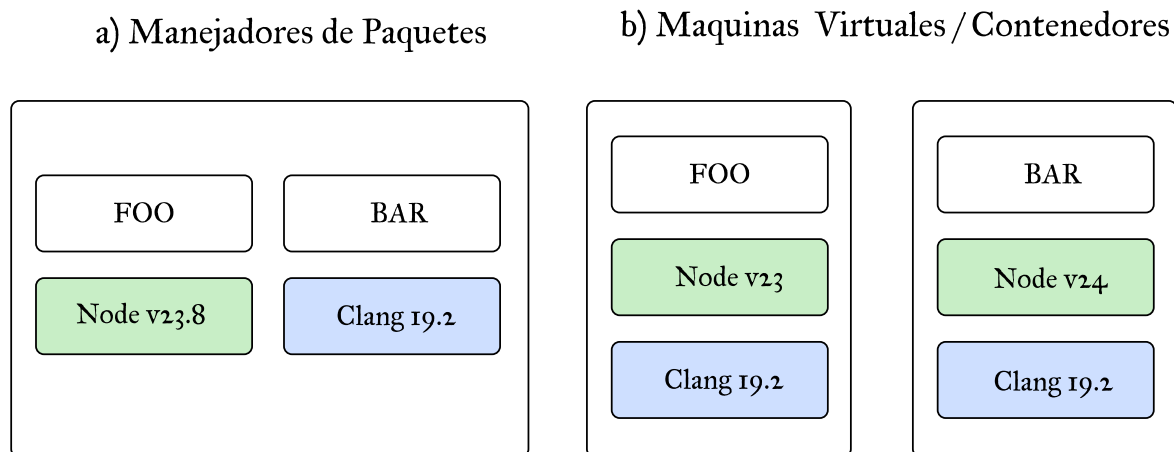


Figura 1: Dependencias de dos programas ficticios F00 y BAR en manejadores de paquetes vs. virtualización.

La unión hace la fuerza, dando origen en 2003 a Nix como una tercera alternativa que fusiona ideas de ambas corrientes partiendo de la idea que: Usar los mismos ingredientes y pasos debería producir el mismo resultado sin importar la computadora. Nix garantiza lo primero mediante identificadores (ID) únicos en un entorno aislado (*Nix Store*) que, como se ve en la Figura 2, permiten la coexistencia de versiones distintas del mismo paquete y la reutilización de dependencias compartidas, resolviendo los problemas previos de F00 y BAR; y lo segundo mediante entornos aislados que aseguran la reproducibilidad [2]. La elegancia de Nix reside en cómo construye estos identificadores y entornos aislados.

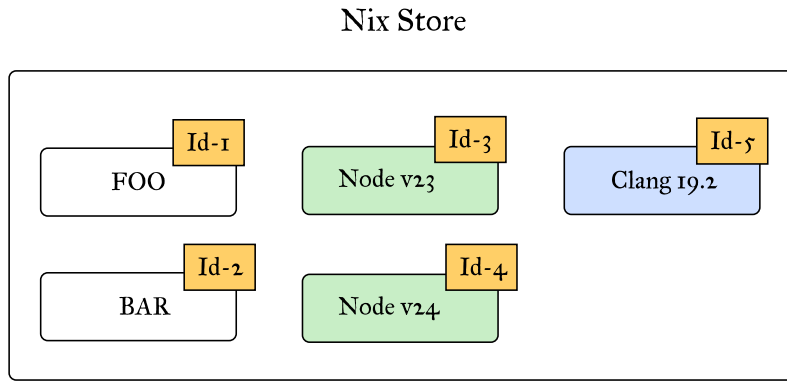


Figura 2: Manejo de paquetes en Nix con identificadores únicos permite coexistencia y reutilización de paquetes.

Fue esta enfoque centrado en seguir recetas explícitas que permitió a Nix conseguir una serie de hitos importantes al contar con unos de los repositorios de paquetes generales más grandes de Linux [8], de los cuales 700 mil han demostrado poder replicarse de forma binariamente idéntica en diferentes computadoras [9]. Aconteció que mucha de las ideas podían generalizarse hasta al punto de reproducir casi por completo un sistema operativo, dando origen a la distribución NixOS [10].

A pesar de ello, Nix ha gozado de una adopción bastante reducida en comparación a las otras herramientas discutidas [11]. ¿Cuál es entonces su talón de Aquiles? Tal parece que no son necesariamente problemas técnicos, sino de experiencia de uso; en encuestas hechas en el foro oficial, la comunidad resaltaba problemas importantes con la documentación, errores crípticos y un *Domain Specific Language* (DSL) difícil de dominar [12], [13]; además, en otra encuesta, se estimó que los usuarios perciben requerir un tiempo de 5 años para dominar la herramienta a pesar que la mayoría lo usa a diario. Llevando a un raro caso donde a pesar que la comunidad le encanta la idea detrás de Nix [14] sus problemas de usabilidad son tan severos que podrían estar impidiendo su uso, lo que concuerda con observaciones de otros estudio en herramientas son situaciones similares [15].

El concepto que los desarrolladores también son usuarios dio origen al campo de estudio de Experiencia de Desarrollo (o DX por sus siglas en inglés), donde el estudio sobre como los desarrolladores perciben sus herramientas ha sido un tema frecuente [16] sobre el que ya se han desarrollado algunos instrumentos como DEXI para evaluar dichas dimensiones [17]. Y dado el trayecto de intentos por mejorar la DX en Nix [18]–[21] el presente trabajo, busca ser una aplicación de las técnicas aprendidas en el campo de DX, en conjunto con el diseño de lenguajes, para evaluar si un Lenguaje de Propósito Específico Embebido (eDSL por sus siglas en inglés) en Typescript [22] podría ayudar a reducir la barra de entrada para nuevos desarrolladores en la herramienta.

3. Objetivos

3.1. General

Evaluar si un eDSL en TypeScript reduce la barrera de entrada a Nix —referente funcional y declarativo en gestión de paquetes, limitado por su curva de aprendizaje— frente a Nixlang, mediante tiempo de completación de tareas y experiencia de usuario.

3.2. Específicos

1. Identificar los principales puntos de dolor cognitivos que presenta el lenguaje de Nix, para fundamentar el diseño de un eDSL, mediante sesiones de pensar-en-alto y «Programación Natural» con estudiantes de Ciencias de la Computación que no hayan utilizado Nix previamente.
2. Desarrollar un eDSL en TypeScript que sirva de prototipo funcional para la evaluación comparativa, capaz de generar archivos de configuración en Nixlang, cubriendo al menos las funcionalidades de la librería estándar, verificado con una batería de pruebas.
3. Comparar Nixlang frente al eDSL desarrollado, para determinar si la familiaridad con Typescript reduce la carga cognitiva de adopción mediante un cuestionario estructurado, y el uso de Short AttrakDiff 2 y DEXI aplicados a estudiantes de Ciencias de la Computación sin experiencia previa, con análisis estadístico Mann-Whitney.

4. Justificación

Con el crecimiento del mercado de los servicios de infraestructura como código [23], se ha aprendido que el poder definir el estado de sistemas completos a través de código, trae ventajas importantes en velocidad de desarrollo, escalabilidad y costos [24]. La misma idea también se ha aplicado a entornos de desarrollo [25] o flujos de despliegue continuo [26]; todo lo anterior sugiere que herramientas que permiten definir entidades o procesos de forma declarativa pueden facilitar el ciclo de desarrollo de *software*. En el manejo de paquetes, el panorama es fragmentado: habiendo alternativas por lenguaje, o indirectas como docker [5]; y una solución declarativa de propósito general no es de el todo clara. Nix lleva años intentando llenar ese espacio — y las cifras sugieren que esta haciendo algo bien: al tener uno de los repositorios de paquetes más grandes de Linux, con 115 mil paquetes [8] un crecimiento del 264% en número de mantenedores en los últimos seis años [27].

Por medio de su lenguaje de configuración, Nix permite describir: la construcción, instalación y composición de paquetes de *software* [2], habilidad que se ha mostrado aplicable en configuración de ambientes de Computación de Alto Rendimiento [28], [29], sistemas operativos [30] despliegue de *software* [31], orquestación de servicios [32] o entornos de desarrollo [33].

No obstante, a pesar de su versatilidad, Nix presenta una barrera de entrada considerable. Reportes sugieren una curva de aprendizaje pronunciada [14], atribuida en parte a la complejidad de su lenguaje de configuración (Nixlang) para ciertos usuarios [21], aspecto que también su creador ha identificado como susceptible de mejora [34]. Estas dificultades podrían estar incidiendo en su adopción relativamente limitada frente a herramientas como Docker [11].

Nixlang es la principal interfaz para interactuar con el ecosistema Nix, es un lenguaje de dominio específico (DSL por sus siglas en inglés) diseñado directamente para expresar los constructos de la herramienta, y es, en gran medida, responsable de la flexibilidad que la caracteriza [35]. Sin embargo, esta misma especialización introduce complejidades que afectan su accesibilidad y usabilidad [34]. Como respuesta, la comunidad ha explorado diversas estrategias para mitigar estas limitaciones, como : extensiones al lenguaje, con la incorporación de tipado estático —esfuerzos que han sido abandonados debido a su complejidad técnica— [18], [20]; agentes de inteligencia artificial para generar configuraciones, aún sin validación empírica sólida en términos de usabilidad [36]; y, en un enfoque más radical, la sustitución del lenguaje por Guile, un lenguaje de propósito general [37], aunque tampoco es un lenguaje muy conocido [11], sabiendo que pertenece a la familia de Lisp [38].

Con base en las propuestas anteriores se hizo un análisis comparativo (véase Sección 8.1), donde se observa que las soluciones existentes tienden a introducir nuevas fuentes de complejidad o dependen de factores externos, sin abordar las causas estructurales de la fricción en Nix; esto sugiere que un posible buen enfoque consistiría en interactuar con Nix usando un lenguaje ampliamente conocido, y Typescript encaja muy bien en ese molde dada su popularidad [11] y similitud sintáctica con Nixlang, siendo descrito en ocasiones como «JSON con funciones» (siendo JSON una notación usada en Typescript) [35].

El uso de lenguajes de propósito general para expresar dominios específicos —conocido como lenguaje de propósito específico embebido (eDSL) [39]— no es un enfoque novedoso y ha demostrado ser efectivo en contextos similares. Un caso ilustrativo es Neovim, que en 2021 introdujo un eDSL en Lua como alternativa a Vimscript [40], lo cual coincidió con un incremento notable en su interés (véase Sección 8.2). En el ecosistema de Nix, existe también una propuesta de eDSL en JavaScript ; no obstante, se trata de un proyecto sin actividad reciente, limitado a un subconjunto de funcionalidades —principalmente la creación de paquetes— y sin evidencia empírica que respalde mejoras en la experiencia de desarrollo [41].

En este contexto, persiste la ausencia de una propuesta que combine un eDSL basado en un lenguaje ampliamente adoptado, con cobertura funcional mas amplia de Nixlang y validación empírica de mejoras en la experiencia de desarrollo.

5. Metodología

La investigación se divide en tres fases ejecutadas de forma secuencial, con el objetivo de desarrollar y evaluar un lenguaje de dominio específico embebido (eDSL, por sus siglas en inglés) en TypeScript como alternativa al lenguaje de configuración original de Nix (en adelante, Nixlang).

5.1. Confidencialidad y Seguridad

Todas las fases que involucran participantes humanos se rigen por los siguientes principios:

- **Participación voluntaria:** La participación es completamente voluntaria. Los participantes pueden retirarse en cualquier momento sin consecuencia alguna.
- **Mayoría de edad:** Todos los participantes deben ser mayores de 18 años.
- **Anonimización:** Los datos recolectados serán disociados de la identidad de los participantes mediante el uso de códigos de identificación internos. Ningún dato publicado o analizado contendrá información que permita identificar a los participantes. Las transcripciones de los fragmentos verbales citados en el análisis serán igualmente anonimizadas.
- **Confidencialidad:** Los datos serán almacenados en dispositivos protegidos con contraseña, accesibles únicamente por el equipo investigador.
- **Destrucción de datos sensibles:** Las grabaciones de pantalla y voz obtenidas durante las sesiones serán destruidas una vez concluido el análisis de datos. Las transcripciones anonimizadas podrán conservarse como parte del registro de investigación.
- **Compensación:** Como agradecimiento por su tiempo, los participantes recibirán una compensación simbólica en forma de un dulce al finalizar la sesión. Esta compensación no condiciona la participación ni sus respuestas.

Antes de empezar todos los participantes se les solicitara aceptar un **Consentimiento informado** en el cual el equipo investigador se compromete a cumplir los puntos anteriores.

5.2. Fase 1: Investigación preliminar

El propósito de la primera fase es identificar los puntos de dolor que cuenta Nixlang, siendo la base para construir una solución los reduzca.

5.2.1. Perfil de los participantes

Se reclután estudiantes de Ciencias de la Computación con poca/nula experiencia en Nix o empaquetado de aplicaciones.

5.2.2. Instrumentos

- **Formulario de perfil:** Recoge datos sobre la experiencia previa del participante con herramientas de gestión de paquetes y lenguajes de programación, así como su edad y semestre cursado.
- **Formulario de programación natural:** Presenta al participante una serie de problemas relacionados con el dominio del empaquetado de aplicaciones, solicitándole que describa con sus propias palabras un algoritmo para resolverlos. Está basado en la técnica de Programación Natural [42], y se entrega de forma impresa junto con hojas en blanco para que el participante responda libremente.
- **Guía rápida de Nix:** Resumen de la sintaxis de Nixlang basado en la documentación oficial [35], presentado después del formulario de programación natural para evitar que el conocimiento del lenguaje influya en las respuestas previas.

- **Guía de actividades de pensar en alto:** Conjunto de tareas a resolver con Nixlang, siguiendo el protocolo de pensar en alto [43].

5.2.3. Procedimiento

Antes de iniciar la sesión, el participante lee y firma el consentimiento informado y completa el formulario de perfil.

A continuación, se entrega el formulario de programación natural de forma impresa. Esta actividad se realiza antes de presentar cualquier material sobre Nixlang, con el propósito de capturar la intuición natural del participante sin sesgo previo de exposición al lenguaje.

Una vez completado el formulario, se presenta la guía rápida de Nix para que el participante pueda familiarizarse con las estructuras y reglas del lenguaje.

Por último, se inicia la sesión de pensar en alto utilizando una computadora provista por el investigador. El participante verbaliza su proceso de pensamiento mientras resuelve las tareas propuestas en Nixlang. Durante esta sesión se graban la pantalla y el audio, previa autorización explícita en el consentimiento informado.

5.2.4. Análisis de datos

Los datos recolectados se analizan con los siguientes objetivos: caracterizar el perfil de los participantes, identificar patrones en sus respuestas de programación natural, y categorizar los puntos de dolor cognitivos observados durante las sesiones de pensar en alto. Los fragmentos verbales más representativos pueden ser citados de forma textual en el análisis, previa anonimización. Los hallazgos de esta fase orientaran el diseño del eDSL en la Fase 2.

5.3. Fase 2: Desarrollo del eDSL

Con base en los hallazgos de la Fase 1, se desarrolló un eDSL en TypeScript capaz de generar archivos de configuración válidos en Nixlang. El diseño del eDSL buscó abordar directamente los puntos de dolor identificados en la fase anterior, cubriendo al menos las funcionalidades de la biblioteca estándar de Nix. Esta fase no involucra participantes humanos y el código fuente se encuentra bajo la licencia MIT [44].

5.4. Fase 3: Evaluación comparativa

La Fase 3 adopta el diseño metodológico de ingeniería de software empírica propuesto por [45], adaptado al contexto de comparación entre un DSL (Nixlang) y un eDSL en TypeScript. La fase se estructura en dos partes: una evaluación de comprensión cognitiva mediante un cuestionario estructurado, y una evaluación de experiencia de desarrollo mediante instrumentos de UX.

5.4.1. Perfil de los participantes

Se reclutan estudiantes de Ciencias de la Computación con poca/nula experiencia en Nix, distintos a los participantes de la Fase 1.

5.4.2. Instrumentos

- **Formulario de perfil:** Idéntico al utilizado en la Fase 1. Incluye una autoevaluación del nivel general de programación, experiencia en TypeScript y familiaridad previa con DSLs.

- **Tutoriales de Nixlang y eDSL:** Presentación del dominio del problema (gestión y empaquetado de aplicaciones) y de la sintaxis de Nixlang y el eDSL desarrollado, con ejemplos representativos.
- **Cuestionarios de comprensión cognitiva:** Instrumento estructurado con preguntas que evaluarán el uso de Nixlang y el eDSL en tres categorías cognitivas basadas en el marco de Dimensiones Cognitivas [45]:
 - **Aprendizaje:** Selección de declaraciones sintácticamente correctas y programas válidos para un resultado dado.
 - **Percepción:** Identificación de constructos del lenguaje y significados correctos de programas.
 - **Evolución:** Preguntas de tipo ensayo donde se solicita al participante expandir, eliminar o modificar la funcionalidad de código existente.

Además también se mide el tiempo de respuesta y tasa de éxito.

- **DEXI (Índice de Experiencia de Desarrollo):** Instrumento para medir la experiencia de desarrollo [17].
- **AttrakDiff-2 corto:** Instrumento para evaluar los aspectos hedónicos y emocionales de la experiencia con ambas herramientas [46].
- **OUX (Evaluación general de la experiencia de usuario):** Cuestionario para medir la percepción general de la experiencia de usuario usando una escala de Likert de 7 puntos basado en [17].

5.4.3. Diseño experimental

Los participantes se dividieron en 2 grupos, la primera mitad interactuando con Nixlang y la segunda con el eDSL desarrollado.

5.4.4. Procedimiento

El participante firma el consentimiento informado y completa el formulario de perfil. A continuación, se le presenta el tutorial correspondiente a la herramienta correspondiente a su grupo. Una vez revisado el material, el participante responde el cuestionario de comprensión cognitiva de esa condición.

Al concluir la herramienta, el participante completa los cuestionarios DEXI, OUX y AttrakDiff. Estos instrumentos se aplican una única vez al final de la sesión, evaluando de forma retrospectiva la experiencia con la herramienta.

5.4.5. Métricas y análisis de datos

Comprensión cognitiva

Para cada cuestionario se calcula la tasa de éxito S_j , definida como el porcentaje promedio de respuestas correctas para cada pregunta j . Las dimensiones cognitivas se evalúan mediante la fórmula:

$$D_i = \sum_{j=1}^N \frac{Q_{ij} \cdot S_j}{C_j}$$

Donde N es la cantidad de participantes, Q_{ij} indica si la dimensión i está asociada a la pregunta j , S_j es la tasa de éxito en la pregunta j , y C_j es el número de dimensiones relevantes para esa pregunta [45]. Las dimensiones evaluadas incluyen: cercanía de mapeo, viscosidad, dependencias ocultas, operaciones mentales difíciles, difusión y expresividad de rol [47].

Experiencia de desarrollo

Las diferencias entre condiciones en los instrumentos DEXI, OUX y AttrakDiff se analizan mediante la prueba no paramétrica de Mann-Whitney, apropiada dado el tamaño reducido de la muestra.

6. Plan de Trabajo

Para poder llevar a cabo la investigación las diferentes fases fueron divididas en tareas más pequeñas y se calendarizaron luego en el siguiente cronograma.

Tarea/ Semanas	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Fase 1 · Investigación preliminar																		
Diseño de instrumentos																		
Reclutamiento participantes																		
Sesiones pensar-en-alto y programación natural																		
Análisis cualitativo de las sesiones																		
Fase 2 · Desarrollo eDSL																		
Diseño arquitectura eDSL																		
Implementación core eDSL																		
Cobertura stdlib Nix																		
Pruebas y validación																		
Fase 3 · Evaluación comparativa																		
Diseño cuestionarios cognitivos																		
Reclutamiento participantes																		
Sesiones evaluación cognitiva																		
Sesiones AttrakDiff / DEXI / OUX																		
Análisis estadístico																		
Síntesis de resultados																		

Tabla 1: Diagrama de Gannt de la ejecución de tareas a lo largo del tiempo

7. Índice preliminar

1. Dedicatorio
2. Resumen/Abstract
3. Tabla de Contenido
4. Introducción
5. Objetivos
6. Marco Teórico
 - 6.1. Despliegue de **software** y sus problemas
 - 6.2. Nix como una solución
 - 6.3. Experiencia de Desarrollo
 - 6.4. Intentos para mejorar Nix
 - 6.5. Transpiladores
7. Metodología
 - 7.1. Fase 1: Investigación preliminar
 - 7.2. Fase 2: Desarrollo del eDSL
 - 7.2.1. Arquitectura
 - 7.3. Fase 3: Evaluación comparativa
 - 7.3.1. Cuestionario estructurado
 - 7.3.2. AttrakDiff-2
 - 7.3.3. DEXI
8. Resultados
9. Discusión
10. Referencias
11. Anexos

Referencias

- [1] M. V. Mäntylä y J. Vanhanen, «Software Deployment Activities and Challenges - A Case Study of Four Software Product Companies», en *2011 15th European Conference on Software Maintenance and Reengineering*, mar. 2011, pp. 131-140. doi: 10.1109/CSMR.2011.19.
- [2] E. Dolstra, «The Purely Functional Deployment Model», phdthesis, 2006. [En línea]. Disponible en: <https://edolstra.github.io/pubs/phd-thesis.pdf>
- [3] R. Gibb, P. Ferris, D. Allsopp, T. Gazagnaire, y A. Madhavapeddy, «Package Managers à La Carte: A Formal Model of Dependency Resolution», *Proceedings of the ACM on Programming Languages*, feb. 2026, [En línea]. Disponible en: <https://arxiv.org/abs/2602.18602v1>
- [4] «(PDF) Infrastructure as Code: Historical Insights and Future Directions», *ResearchGate*, doi: 10.21275/SR24820064820.
- [5] M. Zwinger, «Nix: A Solution with Problems», abr. 2026, [En línea]. Disponible en: <https://arxiv.org/abs/2604.11398v1>
- [6] M. Sobieraj y D. Kotyński, «Docker Performance Evaluation across Operating Systems», *Applied Sciences 2024, Vol. 14, Page 6672*, vol. 14, n.º 15, p. 6672, jul. 2024, doi: 10.3390/APP14156672.
- [7] A. Lingayat, R. R. Badre, y A. K. Gupta, «Performance Evaluation for Deploying Docker Containers on Baremetal and Virtual Machine», *Proceedings of the 3rd International Conference on Communication and Electronics Systems, ICCES 2018*, pp. 1019-1023, oct. 2018, doi: 10.1109/CESYS.2018.8723998.
- [8] D. Marakasov, *Repository Statistics*. Accedido: 16 de abril de 2026. [En línea]. Disponible en: <https://repology.org/repositories/statistics/total>
- [9] J. Malka, S. Zacchiroli, y T. Zimmermann, «Does Functional Package Management Enable Reproducible Builds at Scale? Yes», *Proceedings - 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, MSR 2025*, pp. 775-787, may 2025, doi: 10.1109/MSR66628.2025.00115.
- [10] E. Dolstra y A. Löh, «NixOS: A Purely Functional Linux Distribution», *Proceedings of the ACM SIGPLAN International Conference on Functional Programming, ICFP*, pp. 367-378, sep. 2008, doi: 10.1145/1411204.1411255/SUPPL_FILE/P367-SLIDES.ZIP.
- [11] S. Overflow, *Most Popular Technologies*. Accedido: 16 de abril de 2026. [En línea]. Disponible en: <https://survey.stackoverflow.co/2024/technology>
- [12] «2022 Nix Survey Results - Announcements». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://discourse.nixos.org/t/2022-nix-survey-results/18983>
- [13] «Nix Community Survey 2023 Results - Announcements». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://discourse.nixos.org/t/nix-community-survey-2023-results/33124>
- [14] «Nix Community Survey 2024 Results - Announcements». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://discourse.nixos.org/t/nix-community-survey-2024-results/55403>
- [15] N. C. Goodwin, «Functionality and Usability», *Commun. ACM*, vol. 30, n.º 3, pp. 229-233, mar. 1987, doi: 10.1145/214748.214758.
- [16] A. Razzaq, J. Buckley, Q. Lai, T. Yu, y G. Botterweck, «A Systematic Literature Review on the Influence of Enhanced Developer Experience on Developers' Productivity: Factors, Practices, and Recommendations», *ACM Computing Surveys*, vol. 57, n.º 1, p. 46, oct. 2024, doi: 10.1145/3687299/ASSET/0A1CD69B-DCF1-402A-884C-31D996FEA61F/ASSETS/IMAGES/LARGE/CSUR-2022-0089-F06.JPG.

- [17] K. Kuusinen, «Are Software Developers Just Users of Development Tools? Assessing Developer Experience of a Graphical User Interface Designer», *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, pp. 215-233, ago. 2016, doi: 10.1007/978-3-319-44902-9_14.
- [18] F. Caddet, «Nix with; with Nickel». Accedido: 5 de mayo de 2026. [En línea]. Disponible en: <https://tweag.io/blog/2023-01-24-nix-with-with-nickel/>
- [19] V. Gagarin, «Four Months into The Nix Book». Accedido: 5 de mayo de 2026. [En línea]. Disponible en: <https://tweag.io/blog/2022-09-29-the-nix-book-report/>
- [20] T. Hufschmitt, «Current State of Ptyx». Accedido: 5 de mayo de 2026. [En línea]. Disponible en: <https://github.com/thufschmitt/ptyx/issues/8>
- [21] Valentin Gagarin, «Usability Studies - Development / Documentation - NixOS Discourse». [En línea]. Disponible en: <https://discourse.nixos.org/t/usability-studies/21404>
- [22] «Typescript». Accedido: 2 de mayo de 2026. [En línea]. Disponible en: <https://www.typescriptlang.org/>
- [23] G. V. Research, «Infrastructure As Code Market Size | Industry Report, 2033». Accedido: 6 de mayo de 2026. [En línea]. Disponible en: <https://www.grandviewresearch.com/industry-analysis/infrastructure-as-code-market-report>
- [24] S. Pandya y R. Guha Thakurta, «Introduction to Infrastructure as Code», *Introduction to Infrastructure as Code: A Brief Guide to the Future of DevOps*. Apress, Berkeley, CA, pp. 3-17, 2022. doi: 10.1007/978-1-4842-8689-0_1.
- [25] H. Ghanbari, T. Terimaa, y K. Koskinen, «Using Development Environment as Code for Enhancing Developer Experience: An Action Design Research Study», *Journal of Systems and Software*, vol. 236, p. 112803, jun. 2026, doi: 10.1016/j.jss.2026.112803.
- [26] M. Wessel, J. Vargovich, M. A. Gerosa, y C. Treude, «GitHub Actions: The Impact on the Pull Request Process», *Empirical Software Engineering*, vol. 28, n.º 6, p. 131, nov. 2023, doi: 10.1007/s10664-023-10369-w.
- [27] gg- solutions, «Linux: The Silent Tech Revolution». Accedido: 24 de mayo de 2026. [En línea]. Disponible en: <https://gg-solutions.hashnode.dev/linux-the-silent-revolution>
- [28] Q. Guilloteau, J. Bleuzen, M. Poquet, y O. Richard, «Painless Transposition of Reproducible Distributed Environments with NixOS Compose», en *CLUSTER 2022 - IEEE International Conference on Cluster Computing*, Heidelberg, Germany, sep. 2022, pp. 1-12. Accedido: 6 de mayo de 2026. [En línea]. Disponible en: <https://hal.science/hal-03723771>
- [29] C. E. Gomez, C. J. B. H., y O. Richard, «Impact of Functional Package Manager over a Non-Traditional High Performance Computing System», *2020 IEEE Colombian Conference on Communications and Computing, COLCOM 2020 - Proceedings*, ago. 2020, doi: 10.1109/COLCOM50121.2020.9219768.
- [30] B. Thiberg, «There Are Many like It, but This One Is Mine: Approaches to Personalization of Non-Persistent Development Environments Using Nix», 2025.
- [31] S. V. D. Burg y E. Dolstra, «Disnix: A Toolset for Distributed Deployment», *Science of Computer Programming*, vol. 79, pp. 52-69, ene. 2014, doi: 10.1016/J.SCICO.2012.03.006.
- [32] «Flox | Kubernetes, Uncontained». Accedido: 6 de mayo de 2026. [En línea]. Disponible en: <https://flox.dev/kubernetes/>

- [33] Replit, «Replit — How We Went from Supporting 50 Languages to All of Them». Accedido: 6 de mayo de 2026. [En línea]. Disponible en: <https://blog.replit.com/nix>
- [34] E. Dolstra, «The Future of Nix», 2018.
- [35] «Nix.Dev Documentation». Accedido: 6 de mayo de 2026. [En línea]. Disponible en: <https://nix.dev/tutorials/nix-language>
- [36] M. Schwaighofer, M. Monis, N. Saavedra, J. F. Ferreira, y R. Mayrhofer, «Vibenix: An AI Assistant for Software Packaging with Nix», 2026, [En línea]. Disponible en: <https://www.ndss-symposium.org/wp-content/uploads/lastx2026-35.pdf>
- [37] L. Courtès, «Functional Package Management with Guix», may 2013, [En línea]. Disponible en: <https://arxiv.org/abs/1305.4584v1>
- [38] «Introduction (Guile Reference Manual)». Accedido: 8 de mayo de 2026. [En línea]. Disponible en: https://www.gnu.org/software/guile/manual/html_node/Introduction.html
- [39] A. van Deursen, P. Klint, y J. Visser, «Domain-Specific Languages: An Annotated Bibliography», *SIGPLAN Not.*, vol. 35, n.º 6, pp. 26-36, jun. 2000, doi: 10.1145/352029.352035.
- [40] «Neovim News #11 - Lua Everywhere!». Accedido: 7 de mayo de 2026. [En línea]. Disponible en: <https://neovim.io/news/2021/07/>
- [41] S. van der Burg, «NiJS An Internal DSL for Nix in Javascript». Accedido: 7 de mayo de 2026. [En línea]. Disponible en: <https://github.com/svanderburg/nijs>
- [42] J. F. Pane y B. A. Myers, «(PDF) More Natural Programming Languages and Environments», *ResearchGate*. 2006. doi: 10.1007/1-4020-5386-X_3.
- [43] «(PDF) The Think Aloud Method - A Practical Guide to Modelling Cognitive Processes». Accedido: 12 de mayo de 2026. [En línea]. Disponible en: https://www.researchgate.net/publication/215439100_The_Think_Aloud_Method_-_A_Practical_Guide_to_Modelling_CognitiveProcesses
- [44] «MIT License». Accedido: 12 de mayo de 2026. [En línea]. Disponible en: <https://mit-license.org/>
- [45] «(PDF) Comparison of XAML and C# Forms Using Cognitive Dimensions Framework», en *ResearchGate*, feb. 2026. Accedido: 12 de mayo de 2026. [En línea]. Disponible en: https://www.researchgate.net/publication/229433121_Comparison_of_XAML_and_C_Forms_using_Cognitive_Dimensions_Framework
- [46] «(PDF) Needs, Affect, and Interactive Products-Facets of User Experience», *ResearchGate*, doi: 10.1016/j.intcom.2010.04.002.
- [47] «(PDF) Cognitive Dimensions of Notations: Design Tools for Cognitive Technology», *ResearchGate*, doi: 10.1007/3-540-44617-6_31.
- [48] Google, «Neovim Interest Trends Data». Accedido: 8 de mayo de 2026. [En línea]. Disponible en: <https://github.com/DanielRasho/Thesis/tree/2f294d0bbaa88d5ce046758b6a5750c0476e8d88/Appendix/A>

8. Anexos

8.1. Soluciones existentes a NixLang

	Problema	Soluciones existentes	Limitaciones	Propuesta
[37]	Necesidad de aprender un lenguaje exclusivo de Nix	Uso de lenguajes de propósito general (e.g., Guile en Guix)	Lenguajes poco adoptados mantienen la barrera de entrada [11]	Adoptar un lenguaje ampliamente conocido
[19]	Escasez de documentación y ejemplos	Reescritura de la documentación en un lugar centralizado	Alto costo de mantenimiento y dependencia de aprobación	Que la estructura del código en sí permite su documentación como el tipado estático
[18] [20]	Dependencia de funciones de la librería estándar con muchos parámetros no documentados.	Extensiones al lenguaje para tipado estático	Alta complejidad de implementación	Delegar el tipado a herramientas externas maduras
[36]	Alta complejidad general del ecosistema	Generación de código con Inteligencia Artificial	No aborda causas estructurales; depende de datos de entrenamiento	Evitar soluciones basadas en generación automática

Tabla 2: Una tabla comparativa sobre algunas de las soluciones propuestas por la comunidad de Nix que buscan mejorar la experiencia en Nixlang de forma directa o indirecta, incluyendo sus limitaciones y una propuesta derivada.

8.2. Interés en Neovim

Con datos extraídos de Google Trends para la búsqueda “Neovim - Programa” en su modo clásico, se obtuvieron series históricas correspondientes a los últimos años. Los datos utilizados se muestran en [48].

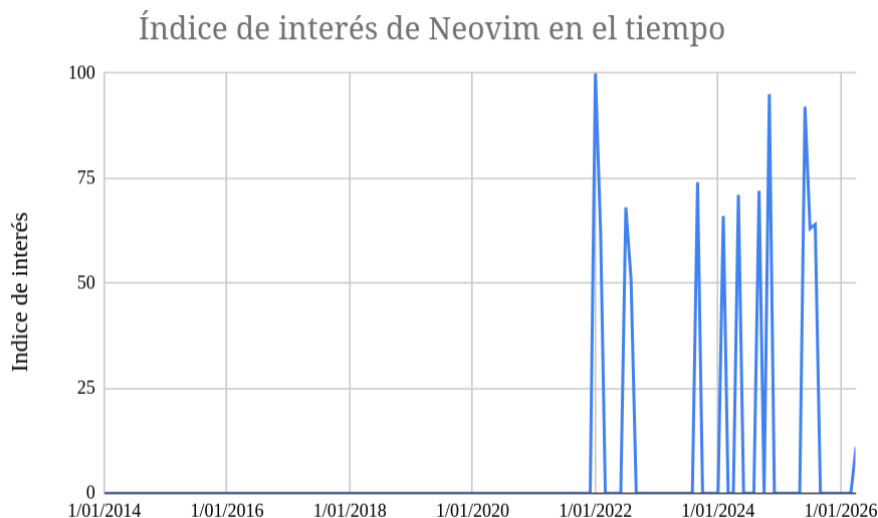


Figura 3: Índice de interés de búsqueda de Neovim en Google entre 2014 y 2026. Se observa un aumento a finales de 2021, aproximadamente seis meses después de la introducción del soporte para Lua [40].